NadiSaathi technical report

OpenCV AI Competition 2026 entry. Numbers generated 2026-10-26 09:47 UTC from `eval/results/*.json` and `bench/results/*.json` by `scripts/make_headline.py`; `make check-numbers` verifies this file against them. Every number below says which data it comes from: REAL (USGS cameras or gauges), SYNTHETIC (procedural scenes with exact truth) or SCENARIO (a replay built from synthetic frames).

One sentence: a camera on the Koshi at Chatara, Nepal, reads the river in metres with OpenCV 5 on AWS Graviton; when the water starts to climb, an agent looks more often, checks itself, works out when the wave reaches Birpur in Bihar, and drafts Hindi, Maithili and Nepali alerts that a named person approves with a PIN.

---

1. Problem and users

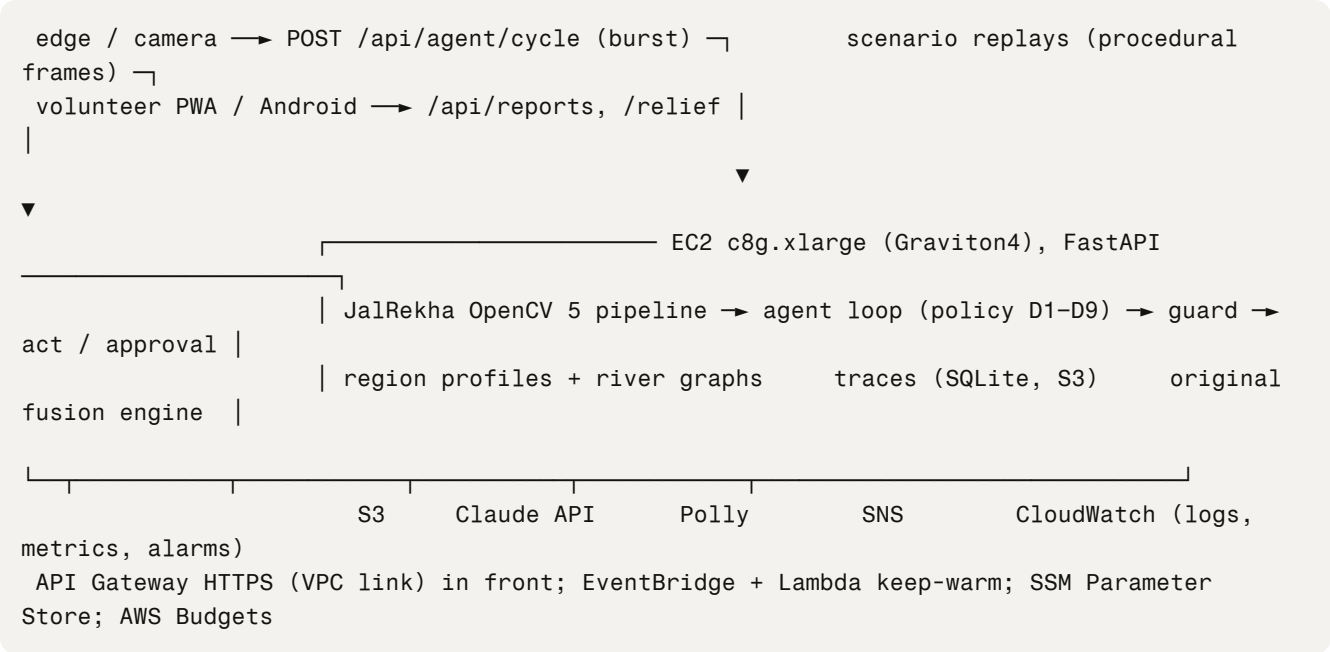
The Koshi, Gandaki/Narayani, Karnali and Bagmati rise in Nepal's hills and flood the plains of Bihar and Uttar Pradesh hours later. On 18 August 2008 the eastern Kosi embankment breached near Kusaha in Nepal and flooded Sunsari and Saptari in Nepal and six districts of Bihar (WRD Bihar). On 29 September 2024 the Kosi at the Birpur barrage peaked at 661,295 cusecs, the highest since 1968 (Government of Bihar). Bihar's flood-affected area is about 17.2% of India's flood-prone area (Government of Bihar). Bihar's own Kosi forecast bulletins in October 2026 note that observed rainfall and water-level data from Nepal were unavailable to them (WRD Bihar). After the August 2026 Bhotekoshi–Trishuli flood, critics said official SMS warnings were generic and lacked exact locations (Mongabay India, secondary). Sources and status for each fact: `docs/FACTS.md`.

Official gauges exist, but between them, on tributaries and at bridges, nobody measures, and the data that do exist do not always cross the border in time. A camera on a staff gauge costs little and can be one more gauge. If it sees the rise upstream, the downstream side gets hours.

SIDE	USER	WHAT NADISAATHI GIVES THEM
Nepal	Ward disaster focal person (rural municipality, Saptari or Sunsari)	the measured level and rise rate in Nepali, a drafted ward SMS and voice clip, one-tap approval
Nepal	Community gauge reader	the camera reads the same gauge every few minutes; asks them for a photo only when it cannot see
India	DDMA control-room operator (Supaul, Saharsa, Khagaria)	an upstream-derived arrival window for named places, a drafted CAP alert and Hindi/Maithili SMS + voice, an approval inbox
India	Panchayat volunteer, NGO relief team	relief requests de-duplicated and ranked; road photos checked for depth; shelter list
Both	Riverside families	a short, plain message in their language, also as audio; a local siren if the cloud is down

The personas in the demo (Sita, Rakesh) are illustrative composites, not real people. No partnership with any agency exists; DHM, CWC, IMD, NDMA, state authorities and community early-warning programmes are the people to approach next.

2. Architecture



Diagrams: `docs/ARCHITECTURE.md` (system and agent workflow, Mermaid). The original stack (FastAPI routes for sites, node analysis, volunteer reports, fused alerts, incidents, sync, CAP; the PWA; the Android app; the Raspberry Pi scripts) is kept and still tested; section 10 lists what existed before and what was built for this competition.

3. OpenCV 5 work

The measurement engine is `backend/src/jalrekha/vision/`, pure OpenCV 5 + NumPy. It runs in the API process on the cloud and out of process on a Raspberry Pi (LiteRT and NumPy in one process corrupted the Vulkan runtime there).

STAGE	FILE	OPENCV USED	OUTPUT
Quality gate	`quality.py`	Laplacian variance, `calcHist` clipping, mean luma, glare fraction, `meanStdDev` contrast on the gauge strip	ok / reasons; a failed frame cannot change an alert
Registration	`register.py`	ORB + `estimateAffinePartial2D` (USAC); escalation to **ALIKED + LightGlue** ONNX models through the OpenCV 5 DNN engine; burst alignment	2×3 transform, inliers, residual; keyframe chaining when high water hides the reference features
Gauge geometry	`gauge.py`	`warpAffine` strip along the gauge axis	piecewise-linear pixel→metre from clicked numerals
Estimator E1	`waterline.py`	Scharr edge energy profile + step fit	waterline row, confidence
Estimator E2	`waterline.py`	**DIS optical flow** temporal texture over the burst	water moves, the gauge does not
Estimator E3	`waterline.py`	Lab colour step fit on the burst median	waterline row
Fusion	`fuse.py`	weighted median, spread → sigma	stage m ± sigma, agreement

STAGE	FILE	OPENCV USED	OUTPUT
Temporal	`temporal.py`	`cv2.KalmanFilter` (level, rate), irregular time steps, Mahalanobis gate	level, rise rate m/h
Surface motion	`flow.py`	DIS flow in the water band	surface speed proxy (not discharge)
Privacy	`privacy.py`	**YuNet** face and licence-plate detectors (`FaceDetectorYN`, DNN), Gaussian blur	redacted frame, counts
Relief photo	`relief.py`	GrabCut seeded by texture, morphology, connected components	water mask, depth at a reference object ± error
Overlay	`overlay.py`	drawing for the UI and traces	annotated JPEG

Vision Lab (`/vision` in the app, `GET /api/vision/lab/{trace_id}`, `vision/explain.py`) shows what OpenCV did for one real agent cycle, stage by stage. It rebuilds the cycle's input frames (scenario frames are regenerated from their seed and cycle time; the Hancock replay reads the archived USGS frame), runs the same `measure()` with an opt-in capture of its intermediates, and returns: each quality metric against its threshold with a |Laplacian| heat map; the ORB (or ALIKED+LightGlue) RANSAC inliers drawn with `cv2.drawMatches` and a checkerboard of the reference against the warped frame; the `warpAffine` gauge strip and its burst median; the per-row signal each estimator fitted, on a gauge axis in metres, with its chosen waterline; the weighted-median fusion, the two-of-three consensus and the agent's Kalman value; and per-stage milliseconds with the host's `cv2.getBuildInformation()` CPU and HAL lines. Every image comes from the YuNet-redacted frame. The page also compares the rebuilt reading with the one stored in the trace; across a full Chatara scenario replay and the full Hancock replay they matched on every cycle checked.

The base method of the original repository (a PIL edge-row heuristic, `services/deterministic_firewall.py`) is kept unchanged as fallback and as baseline B0. It used to report `opencv_used=True` while using PIL; the label is truthful now.

---

4. AWS deployment

One CloudFormation stack, `infra/nadisaathi.yaml`, deploys in about 5 minutes and updates in about 60 s (`make deploy`):

- EC2 c8g.xlarge Graviton4 (Ubuntu 24.04 arm64, stock OpenCV 5 wheel; `USE_COOL_AMI=true` is wired for the COOL AMI but has not been run, because the Marketplace terms are not yet accepted on this account), IMDSv2, no SSH (SSM only), port 8000 reachable only through the API Gateway VPC link.
- API Gateway HTTP API as the public HTTPS front door. CloudFront is in the template; the account must be verified by AWS before it can create CloudFront resources.
- S3: the evidence archive (each agent cycle's JSON event and its face/plate-blurred overlay, `cycles/<site>/<date>/<trace>`, kept 90 days by a lifecycle rule; 575 objects after the replays below), CAP files and deploy bundles. The bucket name contains the account id, so the app serves evidence through its own API instead of presigned links.
- CloudWatch: one JSON event per agent cycle in the stack log group (stage, error, alignment method and inliers, each estimator, every OpenCV stage time, second-opinion model, latency and cost), per-stage OpenCV metrics (`StageMs` by `Stage`), `AgentCycles`, `ApprovalsPending`,

`Escalations`, fixed Logs Insights queries, a dashboard, and alarms for instance health, the public health probe and an OpenCV pipeline slower than 1 s (4 alarms).
- SNS (operator topic; SMS to verified test numbers only), Polly (Hindi/English voice), SSM Parameter Store (operator PINs, camera key, model key), AWS Budgets, an EventBridge-scheduled keep-warm Lambda with a health alarm, and a Step Functions state machine that re-notifies operators every 15 minutes (up to three times) while a drafted public alert is still waiting for a decision. 36 resources in one stack.
- Production numbers, read back from CloudWatch (`scripts/cloud_ops_report.py`): 285 cycles and 1,093 frames over 7 days on 5 camera sites; OpenCV 93 ms per cycle at the median and 308 ms at p95 on Graviton4 (registration 110.6 ms, face/plate blur 53.1 ms, DIS-flow estimator 6.6 ms on average); ORB aligned 271 cycles, ALIKED+LightGlue rescued 3, 11 could not be aligned and were held; 27 live Claude second opinions at 4.1 s median, $0.21 in total. Health probe: 275 probes over 23.0 h, 100.0% up.
- The app's Cloud page (`/cloud`) shows the same data live: the log tail, the Insights queries with their query text, the stage metrics and the state of each service. An MCP server (`make mcp`) exposes the agent's read-only tools and replays to other agents; no tool can approve or send an alert.
- Least-privilege instance role: S3 on the stack bucket, metrics only in the `NadiSaathi` namespace, SNS on the stack topic, logs on the stack log group, and read-only views (metrics, the instance, the stack, escalation runs, the budget) for the Cloud page.
- Cost: c8g.xlarge on-demand is $0.1595/h (AWS Price List API); the judging window is an estimate of about $58 for the whole stack (`docs/DEPLOY.md`).

Language and vision model: Claude (claude-opus-5-5) through the Anthropic API, with server-side refusal fallbacks, structured JSON outputs and a key held in SSM Parameter Store. It writes the second opinion on a gauge reading (D3/D7), structures volunteer reports and relief requests in five languages, describes volunteer photos after face/plate blur, and drafts situation summaries that the numbers-from-tools guard checks. Amazon Bedrock is wired as an alternative provider, but this account's Bedrock quota is zero, so it is not used. Every model call has a deterministic fallback.

On 20 licensed flood photos from India and Nepal (`eval/run_photo_triage.py`, faces and plates blurred first), Claude marked 8 of 8 flooded roads and underpasses as flooded and 2 of the other 12 photos as well (a washed-out mountain bridge and an NDRF boat rescue in floodwater), at ~3.8 s and $0.014 per photo. The labels come from each photo's subject and n is small, so this is a sanity check on real Indian scenes, not an accuracy figure. The photo description never sets a road's status: D8 needs the GrabCut depth check and an operator's approval.

5. The agent

The agent (`backend/src/jalrekha/agent/runner.py`) runs one cycle per camera burst: perceive → assess → plan → guard → act or ask → trace. The OpenCV numbers choose the branch:

BRANCH	TRIGGER (OPENCV OUTPUT)	NEXT ACTION CHANGES TO
D1	quality gate fails	no alert from this frame; burst 5 → 9 frames; photo request after two bad frames

BRANCH	TRIGGER (OPENCV OUTPUT)	NEXT ACTION CHANGES TO
D2	ORB registration fails	ALIKED+LightGlue or keyframe chain; if still lost or the gauge leaves the view, site degraded + recalibration request
D3	fewer than 2 of 3 estimators within 10 cm, or sigma > 8 cm	VLM second opinion (Claude); if unavailable, hold the reading; after 3 held readings in a row, ask a person
D4	significant Kalman rise: rate − 1.5·σ_rate ≥ 0.30 m/h	capture cadence 600 → 300 s (scenario sites); downstream ETA bands
D5	stage ≥ warning, or projected to cross within 1.5 h (right after bad frames: only after 3 significant-rise cycles in a row)	local and cross-border drafts (SMS, voice, CAP) for the right authority role
D6	stage ≥ danger, quality OK, at least 2 of 3 estimators within 10 cm (sigma from the agreeing pair ≤ 8 cm; the trace records both this and the fused sigma)	local siren through the guard (schema, severity ≤ evidence, region bounds, 6/h rate limit)
D7	VLM contradicts a confident reading, is malformed, or carries extra text / instructions	keep the OpenCV value, request human review; a VLM that abstains cannot veto a confident reading
D8	relief photo depth ≥ 0.30 m at a reference object	route-board "blocked" update drafted for approval
D9	falling for 3 cycles after a peak	cadence back down; all-clear proposed for approval

Autonomy policy (`agent/policy.py`, also `GET /api/agent/policy`): the agent alone changes what the camera does, calls the VLM, computes ETAs, drafts messages and sounds the local siren at danger level. Public SMS, voice, CAP, cross-border notices, all-clears, road status and relief dispatch need an operator PIN and a named approver; edits to a draft are kept next to the original text.

Guardrails: strict JSON parsing of VLM replies (only three keys are read; extra prose is reported and ignored), an injection detector, a numbers-must-come-from-tools check for model-written text (`agent/guardrails.py`), the original action guard now wired in with region bounds, fail-safe defaults (a missing reading is "unknown", never "safe"), camera keys on the frame-upload route (a spoofed camera cannot drive the siren), a PIN lockout after five wrong attempts in ten minutes, and API Gateway throttling. A shared operator PIN is a pilot simplification; per-operator credentials are the next step.

The scenario `koshi-night-rise` exercises D1, D2, D4, D5, D6, D7 and D9 (including the all-clear proposal) in one replay; its D7 step uses a scripted adversarial reply, labelled in the trace; `karnali-quiet-storm` checks that bad weather alone raises nothing; `bagmati-camera-moved` checks D2 failure handling. Traces are exported in `docs/traces/`.

6. Evaluation

Protocols are frozen before test runs (`eval/config.frozen.json`, `eval/config.leadtime.frozen.json`; their hashes are stored in each result file). Test sets are held out by scene seed, by site or by date.

6.1 Stage accuracy on real cameras (REAL)

USGS HIVIS cameras at two staff-gauge sites (Hancock, NY on the East Branch Delaware; Moravia, NY on Owasco Inlet), paired with NWIS gauge height within one minute. Calibration is physical: the printed numerals were clicked once on one reference frame, stage = numeral × 0.3048 m, with no fitted parameters, so every frame is a test frame. Single hourly snapshots, so E2 (which needs a burst) abstains.

METHOD	MAE CM (POOLED, 115 IN-RANGE FRAMES)
Full pipeline	**10.6** (95% CI 7.0–16.1), median 4.1, 65% within 10 cm
Full, no registration	10.3 (79% within 10 cm)
E1 edge only	55.2
Naive max-edge (B1)	120.5
E3 colour only	10.6 (65% within 10 cm), identical to the full pipeline: on single snapshots E2 abstains and E1 is down-weighted
Original PIL heuristic (B0)	99.2 on the 38% of frames it answers; on those same 44 frames the full pipeline is 16.5

Per site: Hancock 9.8 cm MAE, median 2.5 cm, 89% within 10 cm (55 frames); Moravia 11.3 cm (60 frames). Frames with the water below the gauge foot or the gauge submerged are scored separately: 19 of 33 (Hancock) and 13 of 15 (Moravia) were reported at the gauge end within 10 cm. A vision-language model (claude-opus-5-5) reading the gauge on the same 115 real frames, told the gauge span and that its numerals are in feet: it answered 72% of frames at 14.0 cm MAE (median 8.4 cm), about 2.5 s and $6.263 per 1,000 frames; on the 83 frames it answered, OpenCV's error is 10.0 cm vs the model's 14.0 cm. A third staff-gauge site (Avon, NY) is out of scope: its plates use another datum and stay above the water for nearly the whole event. Caveat on the protocol: our first protocol (fit a line on each site's first frames) failed on these same frames (87.7 cm, no better than a constant at 90.6 cm); we switched to physical calibration and excluded Avon after seeing that. Nothing in the reported pipeline is fitted to gauge readings, but the choice of protocol was made after the first look. Registration hurt at Moravia (43% within 10 cm with it, 70% without); we report it rather than tune it on the test frames.

6.1b The agent on a real camera (REAL)

`eval/run_real_replay.py` feeds the 88 archived daytime frames of the Hancock camera (July 2026 flood) to the agent in time order, with the physical calibration and illustrative warning/danger marks on the staff gauge (14 ft and 17 ft; not official flood categories). The camera stores one frame per hour, so the temporal estimator cannot run; for such snapshot cameras the agent takes three consecutive frames within 10 cm as the second vote. Results: 64 readings used at 6.5 cm MAE (median 4.0 cm); 27% held (the first frame after each overnight gap, and the turbulent peak), which sent a review request to a person; the danger siren sounded on the first frame above the danger mark (yes); 0 alert drafts on the quiet days; the all-clear was proposed on the recession (yes). The steep rise happened overnight, when this camera records nothing, so the first warning came at the first morning frame: a real limitation of daytime-only cameras, and the reason the agent asks a deployed camera for more frequent bursts during a rise. The snapshot vote was added after the first run of this replay showed the agent holding almost every reading.

6.2 Stage accuracy on held-out synthetic scenes (SYNTHETIC)

2,160 frames, 30 held-out scene seeds, 9 conditions. Full pipeline 8.0 cm MAE (CI 6.9–9.2), median 1.7 cm, 83% within 5 cm. Single estimators: E1 18.4, E2 33.9, E3 11.8 cm; without registration 8.5 cm. Original heuristic: answers 0.1% of frames as shipped; with its border bug fixed, 74.7 cm. Robustness curves for six degradation types are in `eval/results/sweep_synth.json`.

6.3 Lead time gained across a river (REAL)

8 upstream→downstream USGS gauge pairs (Delaware, Susquehanna, Potomac, Juniata, Wabash), 15-minute gauge height, TRAIN 2021–2023, TEST 2024–Oct 2026. The agent's own alert rule (Kalman rate, projected crossing) runs causally on the downstream gauge alone (baseline) and on both gauges (ours). These are U.S. rivers because Nepal and India gauge series are not public; the measurement is of the mechanism, not of the Koshi.

WARNING LEVELS	EVENTS	MEDIAN GAIN	MEAN GAIN	DETECTION BEFORE CROSSING	UPSTREAM FALSE ALERTS / 30 DAYS / PAIR
NWS action stage (pre-registered primary)	68	0 min	4.1 h	7% → 31%	0.19
95th percentile of TRAIN stage	133	8.5 h (CI 330–630 min)	10.2 h	8% → 74%	0.25
95th percentile, camera noise σ = 5 cm	133	13.5 h	14.6 h	8% → 62%	12.40

The window W within which an upstream alert counts is three times the TRAIN travel time, capped at 48 h, so long gains are possible by construction and the false-alert rate is the check on them. The primary result is a null median: with official action stages the upstream gauge often rose without reaching its own action stage, because those stages are set town by town. With levels on the same statistical footing the median gain is 8.5 h. Camera noise makes the rise rate jumpy and multiplies false alerts; the camera-noise "gains" partly come from chance alerts landing in the window and are not claimed.

Version 2 of the rule counts a rise only when it is significant (rate − z·σ_rate ≥ 0.30 m/h). z = 1.5 was chosen on TRAIN only and frozen (`eval/config.leadtime.v2.frozen.json`); TEST had already been seen in v1, so these are optimistic second-look numbers. Camera-noise upstream false alerts fall from 12.40 to 0.50 per 30 days per pair; the median gain with percentile levels stays 8.5 h; with action stages it stays 0 min. The cost: early projected-crossing alerts at the downstream gauge become rarer (downstream-only detection 8% → 1%). The agent uses v2. Details and failure cases: `eval/LEADTIME.md`.

6.4 Agent scenario suite (SYNTHETIC)

36 runs on held-out gauge scenes: 9 floods with rain, glare and camera knocks during the rise, and 27 no-flood runs: bad weather on a steady river, the river held 30 cm below the warning level while glare, a knock, rain/fog and dusk hit the camera, and the camera mount knocked hard while the river sits 50 cm below the warning level. Two baselines use the same OpenCV reading: a threshold agent (no Kalman, fixed 10-minute cadence) and the agent's own projection rule with no quality gate, no consensus check and no second opinion.

	AGENT	PROJECTION RULE, NO GATES	THRESHOLD ONLY
Correct runs	36/36	34/36	35/36
First alert vs true warning crossing (median)	62 min before	56 min	−4 min
False alerts / false sirens / missed sirens	0 / 0 / 0	2 / 0 / 0	1 / 0 / 0

On the 12 runs from gauge scenes added after the last agent change (never used for development): agent 12/12 correct, 0 false-alert runs, median lead 64 min; projection without gates 10/12, 2, 54 min; threshold 11/12, 1, 0 min.

Reading it honestly: the lead time comes from the Kalman projection, which the "no gates" baseline shares. Where the gates matter is false alarms when the camera fails near the warning line. In the camera-knock runs a misregistered frame reads high; the system without gates drafts alerts (2 runs, both on unseen scenes) and the agent does not. The suite records where the plans split: in moved-363 at T+60 min the no-gates system alerts while the agent's trace shows branch D2: "camera moved: site degraded, recalibration requested" (`eval/results/agent_suite.json`, `plan_splits`). The first run of the hard variants found a failure in the agent that the baselines did not have: glare that passed the quality gate biased two readings 40 cm low, and when the glare ended the jump back looked like a 1.2 m/h rise, so the agent drafted alerts (for approval) on a steady river. The fix is a post-gap rule: right after bad frames, a projected crossing needs three significant-rise cycles in a row. The numbers above are after the fix; on the original seeds they are optimistic, which is why the unseen seeds are reported separately. Where the gates matter is false alarms on bad frames near the warning line. The suite found two real bugs during development (scenario reference frames cached per site instead of per scene, which made the agent miss two floods; drafts rewritten with a falling rate after the peak); both are fixed and covered by tests, and the hard near-warning variants were added after a fresh-eyes review.

6.5 Guard against the vision model (SYNTHETIC)

40 scripted VLM replies, each run through a full agent cycle on a danger-level burst (truth above danger, three estimators agreeing): honest confirmations, clean contradictions, prompt injections paraphrased in English, Hindi and Nepali, and malformed output. 32 of 32 bad replies were caught (measurement kept, siren not cancelled) and 8/8 honest confirmations were accepted (`eval/results/guard_eval.json`). Building this set exposed one flaw, now fixed: a VLM that answered "cannot see" made the agent hold a confident danger reading and skip the siren. The guard has a blind spot that the evaluation also measures: in 11 of 90 strong-glare bursts (12%) OpenCV was confidently wrong, reading 0.7–2.5 m too low with two estimators agreeing, so no second opinion is requested and the low value would stand. These are scripted inputs, not a live model; the injection detector is a pattern list, but the guard does not rely on it: only three JSON keys are ever read, and any contradiction of a confident measurement goes to a person.

6.6 Performance and cost (benchmark workload)

`bench/cool_bench.py`, same frames and config on every host; full table in `docs/COOL.md`. Graviton4 c8g.xlarge with the stock OpenCV 5.0 wheel: 87.0 ms per 5-frame burst (sd 0.19 over 5 repeats), 99.4 frames/s with one process per vCPU, 198 cameras at a 10 s cadence, $0.446 per million frames. x86 c7i.xlarge: 92.5 ms, 93.3 frames/s, $0.531. Outputs are identical to the millimetre across the two

architectures. ALIKED+LightGlue is 1.47× and DIS flow 1.53× faster on Graviton. The aarch64 opencv-python 5.0 wheel already reports KleidiCV in its build information.

7. Reliability

- Tests: backend pytest (agent scenarios, approvals and PIN, guardrails, relief, photo check, lead-time code, and all the original tests), frontend vitest, ESLint, all in `make test`.
- Nothing lost: `make smoke` calls every original API route and the new ones against the deployed stack (34 of 34 as expected on the last run, including the original sample-video analysis and the PIN-protected writes).
- Repeatability: benchmark repeats have a standard deviation of 0.19 ms on c8g; scenario replays are deterministic (seeded).
- The deployed endpoint has a keep-warm Lambda every 5 minutes, an EC2 status-check auto-recover alarm, and a health alarm to SNS. Observed so far: 100.0% of 275 probes OK over 23.0 h, health latency p50 125 ms / p95 232 ms, 1,870 agent cycles run on the cloud host (`scripts/soak_report.py` reads CloudWatch; the number grows through judging).

8. Limitations

- Two real camera sites, both in the United States, both with staff gauges. No Nepal or India camera footage was available; the Koshi, Gandak, Karnali and Bagmati runs are scenario replays and are labelled so.
- Physical calibration assumes the staff gauge shares the official gauge datum (true at both evaluated sites; Avon shows it is not always true).
- Hourly snapshots only on the real cameras; the burst estimator E2 and the cadence logic are evaluated on synthetic scenes.
- Travel times are distance/celerity estimates except one reported value (Devghat–Valmikinagar about six hours, a secondary source).
- Lead-time gains depend on consistent warning levels along the river (6.3).
- No Polly voice for Nepali, Maithili or Bhojpuri; Maithili and Bhojpuri texts need native review.
- SMS to India needs TRAI DLT registration; Nepal bulk SMS goes through telecom operators. The pilot sends only to verified test numbers.
- The language and vision model is a hosted API (Claude); the edge box falls back to rules (or an on-device Gemma model) when it is offline.
- Face and plate redaction recall has not been measured on our own data; we rely on the detectors' published behaviour, and stored frames, approvals and traces are readable through the public demo API (they contain synthetic scenes and no personal data). A real deployment should restrict read access to operators.

9. Responsible use

Faces and number plates are blurred before storage; the agent never tracks people. Relief requests keep text, a coarse location (3 decimal places) and parsed fields, no names or phone numbers. Every public message is approved by a named person with a PIN, and the CAP file carries a note that it is a

pilot and complements official warnings. Unknown is never reported as safe. Model cards: `docs/MODEL_CARDS.md`.

10. What existed before and what was built for this competition

The repository builds on an earlier open-source flood-alert stack. The table separates what existed before from what was built for this competition.

AREA	BEFORE	BUILT FOR THIS COMPETITION
Vision	PIL edge-row heuristic; a language model reading frames	OpenCV 5 metric pipeline (quality, ORB/ALIKED+LightGlue registration, three estimators, Kalman, DIS flow, YuNet redaction, relief photo check)
Region	one country, Spanish	Nepal and India profiles, four basin graphs with CWC levels, five languages, voice
Agent	language model chooses actuators; guard not wired	perception-driven policy D1–D9, approvals, wired guard, guardrails, traces, scenario replays
Cloud	none	CloudFormation on Graviton, API Gateway, S3, Polly, SNS, Claude API (Bedrock wired as alternative), CloudWatch, Budgets, keep-warm
Evaluation	parsing accuracy on authored sentences; Pi latency	real-camera stage accuracy, synthetic robustness, real lead time, agent suite, Graviton/x86 benchmark
UI	dashboard, report, queue, calibration, settings	Live replay, traces, approvals, rivers, relief, story; EN/HI/NE; accessibility fixes
Kept	all original API routes, fusion engine, incidents, sync, CAP, Android app, Pi scripts	still tested